Policy Library CBF: Finite-Horizon Safety at Runtime via Parallel Rollouts

Taekyung Kim[1], Hideki Okamoto[2], Bardh Hoxha[2], Georgios Fainekos[2], Dimitra Panagou[1,3]

***Abstract*— Safety-critical autonomy in unstructured environments poses significant challenges for online safety certification under evolving constraints. We propose Policy Library Control Barrier Function (PL-CBF), a runtime safety filter that evaluates a library of fallback policies via parallel finite-horizon rollouts, selects the least invasive safe mode, and enforces safety by solving a quadratic program that minimally modifies a nominal policy. We provide a theoretical analysis based on a finite-horizon language metric over closed-loop behaviors, characterizing policy-library coverage requirements for certifying finite-horizon safety. Simulations on a planar double-integrator (4 states), highway driving with abrupt friction changes using a realistic nonlinear vehicle model (8 states), and 3D quadrotor navigation in crowded dynamic environments (12 states) demonstrate improved safety coverage over single-policy safety filters while retaining millisecond-level runtime. [Code] [Project Page][1] [Video]**

I. INTRODUCTION

The deployment of autonomous systems in unstructured and real-world environments represents one of the most significant engineering challenges of the modern era. In contrast to classical settings where constraints and model parameters are fixed and fully modeled, a deployed robot operates under *perception-limited* information: the safe area is revealed online [1]–[3], dynamic obstacles appear and disappear from view [4], [5], and operating conditions may change over time [6]–[8]. Consequently, the relevant safety objective is not merely stability with respect to a known invariant set, but the ability to ensure constraint satisfaction in real time under the latest perceived model and constraints.

Formal verification tools such as Hamilton-Jacobi (HJ) reachability analysis provide strong safety guarantees by computing the viability kernel [9]. However, these approaches typically face the curse of dimensionality and become computationally intractable for the high-dimensional systems and complex constraints that arise in robotics. Control Barrier Functions (CBFs) provide a more scalable alternative by enforcing pointwise constraints on the control input [10]. Yet, standard CBF synthesis often presumes access to an explicit barrier function for the full nonlinear system under a fixed constraint description, which is difficult to obtain in general and may not remain valid as perception updates change the effective constraints online [11].

[1]Department of Robotics, [3]Department of Aerospace Engineering, University of Michigan, Ann Arbor, MI, 48109, USA taekyung@umich.edu, dpanagou@umich.edu

[2]Toyota Motor North America, Research & Development, Ann Arbor, MI, 48105, USA <first_name.last_name>@toyota.com

[1]Project page: https://www.taekyung.me/plcbf

A complementary line of work addresses the feasibility aspect by coupling safety filtering with a certified recovery maneuver. In particular, Backup CBFs [12] enforce recursive feasibility by leveraging a designated backup policy and a terminal controlled invariant set, so that the closed loop can be driven to a region where safety can be maintained indefinitely. While conceptually appealing, constructing such a backup policy together with a corresponding terminal invariant set for complex nonlinear dynamics is often difficult in practice, especially when the operational envelope and constraints depend on online perception.

More recently, rollout-based safety filters have been proposed to mitigate the need for offline barrier synthesis. Policy CBFs (PCBFs) construct finite-horizon safety certificates directly from trajectory rollouts [13], [14], enabling runtime certificate construction without offline value-function computation. However, existing runtime rollout-based filters still hinge on the validity of a *single* designated fallback policy. In unstructured settings, no single maneuver is reliably safe across all perception updates: for example, a stop maneuver can fail under sudden friction loss, and a fixed evasive maneuver can become infeasible when newly detected obstacles or narrow passages invalidate that behavior. This single-fallback dependence induces conservatism and can lead to failures even when an alternative safe maneuver exists.

In this paper, we propose the *Policy Library Control Barrier Function* (PL-CBF), a runtime safety filter that retains multimodal safe maneuvers through a finite library of diverse fallback policies. PL-CBF certifies safety whenever at least one library policy remains safe over a planning horizon under the latest perceived constraints. The final safe control input is obtained by solving a Quadratic Program (QP) that minimally modifies the nominal command.

The main contributions of this paper are:

- **Policy-library finite-horizon safety filtering:** We propose PL-CBF, a runtime safety filter that certifies finite-horizon safety by evaluating a finite library of candidate fallback policies via parallel rollouts. Among the certified candidates, PL-CBF selects the least restrictive constraint by maximizing the volume of the admissible control set.
- **Theoretical characterization:** Using a finite-horizon language metric over closed-loop rollouts, we establish a completeness condition for finite-horizon safety certification. We also discuss the conceptual connection to classical CBF and forward-invariance results in an idealized infinite-horizon limit.
- **Empirical validation and runtime:** We evaluate PL-

CBF on three simulation benchmarks (double integrator, highway driving with abrupt friction changes, and 3D quadrotor navigation with limited sensing and dynamic obstacles), demonstrating improved safety coverage over single-policy safety filters while maintaining millisecond-level runtime.

II. BACKGROUND INFORMATION

A. Control Barrier Function

Consider the continuous-time control-affine system

$$\dot{\boldsymbol{x}}(t) = f(\boldsymbol{x}(t)) + g(\boldsymbol{x}(t))\boldsymbol{u}(t), \tag{1}$$

where $\boldsymbol{x}(t) \in \mathcal{X}$ and $\boldsymbol{u}(t) \in \mathcal{U} \subset \mathbb{R}^m$. We assume $\mathcal{U}$ is compact and convex, and that $f : \mathcal{X} \to \mathbb{R}^n$ and $g : \mathcal{X} \to \mathbb{R}^{n\times m}$ are locally Lipschitz continuous. Given a continuously differentiable function $h : \mathcal{X} \to \mathbb{R}$, define the safe set

$$\mathcal{C} \coloneqq \{\boldsymbol{x} \in \mathcal{X} \mid h(\boldsymbol{x}) \geq 0\}. \tag{2}$$

Let $\varphi_t^\pi(\boldsymbol{x}_0)$ denote the state at time $t \geq 0$ of the closed-loop system $\dot{\boldsymbol{x}} = f(\boldsymbol{x}) + g(\boldsymbol{x})\pi(\boldsymbol{x})$ with initial condition $\boldsymbol{x}(0) = \boldsymbol{x}_0$ under a locally Lipschitz feedback policy $\boldsymbol{u} = \pi(\boldsymbol{x})$.

Definition 1 (CBF [10])**.** *A continuously differentiable function $h : \mathcal{X} \to \mathbb{R}$ is a CBF on $\mathcal{C}$ for System* (1) *if there exists an extended class $\mathcal{K}_\infty$ function α such that*

$$\sup_{\boldsymbol{u}\in\mathcal{U}} [L_f h(\boldsymbol{x}) + L_g h(\boldsymbol{x})\boldsymbol{u}] \geq -\alpha(h(\boldsymbol{x})) \quad \forall \boldsymbol{x} \in \mathcal{C}. \tag{3}$$

Given a CBF h, define the CBF-admissible control set

$$K_{\text{cbf}}(\boldsymbol{x}) \coloneqq \{\boldsymbol{u} \in \mathcal{U} \mid L_f h(\boldsymbol{x}) + L_g h(\boldsymbol{x})\boldsymbol{u} + \alpha(h(\boldsymbol{x})) \geq 0\}.$$

If h is a CBF, then any locally Lipschitz controller $\boldsymbol{u} = \pi(\boldsymbol{x})$ satisfying $\pi(\boldsymbol{x}) \in K_{\text{cbf}}(\boldsymbol{x})$ for all $\boldsymbol{x} \in \mathcal{C}$ renders $\mathcal{C}$ controlled invariant, equivalently $\boldsymbol{x}(0) \in \mathcal{C}$ implies $\varphi_t^\pi(\boldsymbol{x}(0)) \in \mathcal{C}$ for all $t \geq 0$ [11].

B. Policy Control Barrier Function

Since computing infinite-horizon rollouts is computationally intractable for real-time applications, a finite-horizon approximation over a horizon T is typically utilized [14]:

$$H_T^\pi(\boldsymbol{x}) \coloneqq \inf_{t\in[0,T]} h(\varphi_t^\pi(\boldsymbol{x})). \tag{4}$$

The validity of this finite-time approximation as a safety certificate depends on the properties of the policy π. We formally distinguish between a *backup policy* and a general *fallback policy*. While a fallback policy is merely a candidate control law that attempts to maintain safety, a backup policy π_{b} provides a strict guarantee of safety and convergence to a known terminal controlled invariant set $\Phi_{\pi_{\text{b}}} \subseteq \mathcal{C}$:

Definition 2 (Backup Policy)**.** *A policy π_{b} is a backup policy if there exists a known controlled invariant set $\Phi_{\pi_{\text{b}}} \subseteq \mathcal{C}$ such that, for any initial state $\boldsymbol{x} \in \mathcal{C}$, the closed-loop trajectory satisfies $\varphi_t^{\pi_{\text{b}}}(\boldsymbol{x}) \in \mathcal{C}$ for all $t \in [0, T]$ and $\varphi_T^{\pi_{\text{b}}}(\boldsymbol{x}) \in \Phi_{\pi_{\text{b}}}$.*

If we choose the backup policy (i.e., $\pi = \pi_{\text{b}}$), the finite-horizon value function coincides with its infinite-horizon counterpart, such that $H_T^{\pi_{\text{b}}}(\boldsymbol{x}) = H_\infty^{\pi_{\text{b}}}(\boldsymbol{x}) \coloneqq \inf_{t\geq 0} h(\varphi_t^{\pi_{\text{b}}}(\boldsymbol{x}))$. This allows the PCBF to recover the recursive feasibility guarantees of Backup CBF approaches. However, in environments where the safe set is revealed online, identifying a valid terminal controlled invariant set $\Phi_{\pi_{\text{b}}}$ *a priori* is often impossible. In the absence of a backup policy π_{b}, the PCBF framework proposes the use of a *fallback policy* [14], which can essentially be any control policy. While this approach does not ensure convergence to a known terminal controlled invariant set $\Phi_{\pi_{\text{b}}}$, it provides a guarantee of safety for the duration of the finite horizon.

Limitation: The limitation of the standard PCBF is its reliance on the validity of ***one*** specific policy π. The safety filter enforces constraints based solely on the closed-loop behavior of this single policy. If the pre-defined policy π is mismatched to the current perceived operating conditions and therefore cannot satisfy the safety constraints, then the filter deems the state unsafe even if an alternative admissible maneuver is feasible. This creates unnecessary conservatism and may lead to safety violations that could have been avoided by a different feasible policy.

III. PROBLEM FORMULATION

A. Discrete Perception Updates and Perceived Safe Set

Let $\{t_k\}_{k\in\mathbb{N}}$ denote the discrete sensing-update times. We define the index map

$$\kappa : \mathbb{R}_{\geq 0} \to \mathbb{N}, \quad \kappa(t) = k \text{ such that } t_k \leq t < t_{k+1}, \tag{5}$$

which returns the latest sensing update available at continuous time t.

At each update time t_k, the robot constructs a *perceived safe set* $\mathcal{C}_k \subset \mathcal{X}$ from the latest environment estimate (e.g., LiDAR-based obstacle detections and occupancy mapping). We represent this set as the 0-superlevel set of a continuously differentiable function $h_k : \mathcal{X} \to \mathbb{R}$:

$$\mathcal{C}_k = \{\boldsymbol{x} \in \mathcal{X} \mid h_k(\boldsymbol{x}) \geq 0\}. \tag{6}$$

We do not impose a monotonicity assumption on the sequence $\{\mathcal{C}_k\}_{k\in\mathbb{N}}$: newly observed free space may enlarge the certified region, whereas newly detected obstacles or occlusions may invalidate regions previously regarded as safe.

Using the same notation $\varphi_{(\cdot)}^\pi(\cdot)$ introduced in Sec. II-A, the look-ahead state at time $t+\tau$ from the current state $\boldsymbol{x}(t)$ under policy π is $\varphi_\tau^\pi(\boldsymbol{x}(t))$, for all $\tau \geq 0$.

B. Problem Statement

Our primary objective is to design a safety filter that tracks a desired nominal policy π_{nom} while ensuring that the system remains within the perceived safe set. Because safety must be certified online using only the current perceived safe set $\mathcal{C}_k$ available over the sensing interval indexed by $k = \kappa(t)$, infinite-horizon safety certification is generally not theoretically possible without strong assumptions in this setting. We therefore consider a finite-horizon safety problem over a planning horizon T.

Problem 1 (Finite-Horizon Safety)**.** At time t, let $k = \kappa(t)$. Given the current perceived safe set $\mathcal{C}_k$, synthesize a control policy π close to π_{nom} such that

$$\varphi_\tau^\pi\big(\boldsymbol{x}(t)\big) \in \mathcal{C}_k, \quad \forall \tau \in [0, T]. \tag{7}$$

Solving Problem 1 recursively across sensing updates allows the robot to navigate safely through the currently observed environment while continuously incorporating new perceived safety constraints $\{h_k\}$.

Remark 1 (Connections to Model Predictive Control (MPC))**.** The finite-horizon safety formulation in Problem 1 is closely related to MPC practices. Theoretically, recursive feasibility in finite-horizon MPC often requires a terminal constraint set. However, in environments where such a terminal set is difficult to synthesize online, practitioners often omit terminal constraints as the resulting MPC remains highly effective in practice [15], [16]. Our formulation which drops the requirement for certifying infinite-time invariance is analogous to dropping the terminal constraint in MPC. However, a key distinction lies in the computational objective. While MPC typically requires solving a computationally expensive optimization problem online, our goal is to design a safety filter that is computationally efficient to construct a safety certificate at runtime.

IV. POLICY LIBRARY CBF

A. The Policy Library CBF Formulation

To overcome the limitations of relying on a single fallback maneuver, we propose the Policy Library Control Barrier Function (PL-CBF). Instead of relying on a single backup or fallback policy, we define a library of K candidate fallback policies

$$\Pi = \{\pi_1, \pi_2, \ldots, \pi_K\}. \tag{8}$$

We then define the finite-horizon *policy library value function*[2]

$$\begin{aligned} H_T^\Pi(\boldsymbol{x}) &\coloneqq \max_{\pi_i \in \Pi} \{H_T^{\pi_i}(\boldsymbol{x})\} \\ &= \max_{\pi_i \in \Pi} \left\{ \inf_{\tau \in [0,T]} h\left(\varphi_\tau^{\pi_i}(\boldsymbol{x})\right) \right\}. \end{aligned} \tag{9}$$

Conceptually, if any policy in the library Π can maintain non-negative constraint values over $[0, T]$, then $\boldsymbol{x}$ is considered safe for horizon T. This induces the *PL-CBF safe set*

$$\mathcal{S}_T^\Pi \coloneqq \bigcup_{\pi_i \in \Pi} \{\boldsymbol{x} \in \mathcal{X} \mid H_T^{\pi_i}(\boldsymbol{x}) \geq 0\}. \tag{10}$$

We emphasize that the explicit construction of $\mathcal{S}_T^\Pi$ is used solely for theoretical analysis to characterize how the certified safe set evolves as the policy library Π changes. In the runtime algorithm, we ***do not*** compute the boundary of $\mathcal{S}_T^\Pi$ or perform operations over the entire state space $\mathcal{X}$. At runtime, we only evaluate $H_T^\Pi(\boldsymbol{x})$ at the current system state $\boldsymbol{x}$. We defer a discussion of the (idealized) infinite-horizon limit and its relation to CBFs and forward invariance to Sec. V-C.

[2]For the remainder of this paper, we omit the subscript k and denote the current perception snapshot h_k simply by h when the meaning is clear.

Algorithm 1: Policy Library CBF QP

Input: Current time, state, and perceived constraint function $\{t, \boldsymbol{x}, h_k\}$, Nominal input $\boldsymbol{u}_{\text{nom}}$, Policy library Π
Output: Safe control input $\boldsymbol{u}^\star$

for $\pi_i \in \Pi$ **do**
  `// Parallelizable`
  Rollout π_i to compute closed-loop trajectory $\varphi_{(\cdot)}^{\pi_i}(\boldsymbol{x}(t))$
  Compute $H_T^{\pi_i}(\boldsymbol{x}) = \inf_{\tau \in [0,T]} h_k(\varphi_\tau^{\pi_i}(\boldsymbol{x}(t)))$
end
Select least invasive policy $\pi_{i^\star} \leftarrow \max_{\pi_i \in \Pi} \text{Vol}\left(K_{\text{cbf}}(\boldsymbol{x}; \pi_i)\right)$
Compute the gradient $\nabla H_T^{\pi_{i^\star}}(\boldsymbol{x})$ using `autodiff`
$\boldsymbol{u}^\star \leftarrow$ solve QP (13) using the constraint from $\pi_{i^\star}$
return $\boldsymbol{u}^\star$

B. Safety Filter Synthesis

We synthesize the safety filter using a QP that minimally deviates from the nominal control $\boldsymbol{u}_{\text{nom}} = \pi_{\text{nom}}(\boldsymbol{x})$. To minimize the invasiveness of the safety filter, we propose selecting the active policy based on the volume of the admissible control space. While standard approaches might select the policy yielding the highest safety value, a higher safety margin does not necessarily imply a less restrictive constraint on the control input. For each policy $\pi_i \in \Pi$, the set of safe control inputs is defined by the corresponding CBF constraint:

$$\begin{aligned} K_{\text{cbf}}(\boldsymbol{x}; \pi_i) = \{\boldsymbol{u} \in \mathcal{U} \mid \nabla H_T^{\pi_i}(\boldsymbol{x})^\top (f(\boldsymbol{x}) + g(\boldsymbol{x})\boldsymbol{u}) \\ \geq -\alpha(H_T^{\pi_i}(\boldsymbol{x}))\}. \end{aligned} \tag{11}$$

We select the least invasive policy index $i^\star$ that maximizes the volume of this admissible set:

$$\begin{aligned} i^\star &= \arg\max_{i \in \mathcal{I}_{\text{safe}}(\boldsymbol{x})} \text{Vol}\left(K_{\text{cbf}}(\boldsymbol{x}; \pi_i)\right), \\ \mathcal{I}_{\text{safe}}(\boldsymbol{x}) &\coloneqq \{\, i \in \{1, \ldots, K\} \mid H_T^{\pi_i}(\boldsymbol{x}) > 0 \,\}. \end{aligned} \tag{12}$$

The final control input is obtained by solving the following QP (see Alg. 1):

$$\begin{aligned} \boldsymbol{u}^\star = \arg\min_{\boldsymbol{u} \in \mathcal{U}} \quad & \|\boldsymbol{u} - \boldsymbol{u}_{\text{nom}}\|_2^2 \\ \text{s.t.} \quad & \nabla H_T^{\pi_{i^\star}}(\boldsymbol{x})^\top (f(\boldsymbol{x}) + g(\boldsymbol{x})\boldsymbol{u}) \geq -\alpha(H_T^{\pi_{i^\star}}(\boldsymbol{x})). \end{aligned} \tag{13}$$

To construct the constraint in (13), we require the gradient of the value function, i.e., $\nabla H_T^{\pi_{i^\star}}(\boldsymbol{x})$. Since the value function is defined by the rollout of nonlinear dynamics, analytical gradients are generally unavailable. Therefore, we utilize automatic differentiation through the dynamics to compute the gradient efficiently at runtime [17]. To mitigate gradient errors arising from discrete-time integration, cubic spline interpolation can be applied to the rollout trajectory as discussed in [14].

C. Parallel Computation and Runtime Efficiency

A significant advantage of the proposed PL-CBF framework is its computational structure which is naturally amenable to parallelization. Unlike HJ reachability analysis which suffers from the curse of dimensionality due to spatial gridding, our method relies on independent trajectory rollouts. The computation of the closed-loop trajectory $\varphi_{(\cdot)}^{\pi_i}(\boldsymbol{x})$ and policy value function $H_T^{\pi_i}(\boldsymbol{x})$ for each policy $\pi_i \in \Pi$

is decoupled and can be executed simultaneously on parallel computing hardware such as GPUs.

This parallel architecture allows the safety filter to scale efficiently with the size of the policy library K. By constructing the safety certificate online at runtime, PL-CBF avoids offline synthesis tied to a single fixed model or constraint realization. Its effectiveness therefore depends on the suitability and diversity of the policy library Π for the runtime scenarios encountered.

V. THEORETICAL ANALYSIS

A. Language Metric and Approximate Bisimulation

We formalize similarity between finite-horizon closed-loop trajectories induced by different fallback policies using language and approximate (bi)simulation metrics from the verification literature on metric transition systems [18], [19].

The finite-horizon trajectory is $\tau \mapsto \varphi_\tau^\pi(\boldsymbol{x})$ for $\tau \in [0, T]$. In the general framework, the *language* of a system is the set of external trajectories it can generate, and *language metrics* are Hausdorff distances between such trajectory sets induced by a trajectory metric [19, Def. 2].

Definition 3 (Finite-Horizon Language Metric [3])**.** *Given an initial condition $\boldsymbol{x}$ and a horizon T, we define the finite-horizon language metric between two policies π_i and π_j by*

$$d_{\boldsymbol{x}}(\pi_i, \pi_j) \coloneqq \sup_{\tau \in [0,T]} \|\varphi_\tau^{\pi_i}(\boldsymbol{x}) - \varphi_\tau^{\pi_j}(\boldsymbol{x})\|_2. \tag{14}$$

The language metric $d_{\boldsymbol{x}}$ upper bounds the worst-case state deviation between two policy rollouts over $[0, T]$, enabling transfer of finite-horizon safety margins via Lipschitz continuity of h (see Sec. V-B). For deterministic closed-loop systems, the bisimulation metric coincides with the language metric [18, Thm 10]. Accordingly, in our problem, bounds expressed in terms of $d_{\boldsymbol{x}}(\cdot, \cdot)$ can also be interpreted through the lens of approximate (bi)simulation.

Definition 4 (Policy-Library Approximate Precision)**.** *Let Π_{all} denote the set of admissible policies that are well-posed for System* (1) *and let $\Pi \subset \Pi_{\text{all}}$ be a finite library. The approximate precision of Π at $\boldsymbol{x}$ under the language (and bisimulation) metric is defined by the directed Hausdorff distance:*

$$\delta_{\boldsymbol{x}}(\Pi) \coloneqq \sup_{\pi^\star \in \Pi_{\text{all}}} \left[\min_{\pi_k \in \Pi} d_{\boldsymbol{x}}(\pi^\star, \pi_k) \right]. \tag{15}$$

Intuitively, for any admissible policy $\pi^\star \in \Pi_{\text{all}}$, there exists a policy $\pi_k \in \Pi$ whose closed-loop trajectory stays within $\delta_{\boldsymbol{x}}(\Pi)$, measured by $d_{\boldsymbol{x}}$, of the trajectory induced by $\pi^\star$ over the horizon $[0, T]$. Therefore, smaller $\delta_{\boldsymbol{x}}(\Pi)$ corresponds to finer coverage of admissible closed-loop behaviors by the finite policy library.

B. Completeness

Safety filters based on a backup [2], [12], [21] or a fallback policy [13], [14] certify safety by validating a *particular* closed-loop controller. Such certificates can be effective when the perceived constraints function h is fixed. In our setting, however, the perception information h_k updates online, so a controller that is safe at time t_k may fail to remain safe after the next update. This motivates the central question addressed by PL-CBF: can a finite library of fallback policies certify safety whenever there exists at least one admissible policy that is safe over the horizon under the current perception information h_k?

We now provide a sufficient condition under which PL-CBF certifies finite-horizon safety whenever a feasible safety maneuver exists for the current snapshot. This result can be interpreted as a *completeness* guarantee: if there exists an admissible policy that is strictly safe with a nonzero margin, and the policy library approximates admissible closed-loop trajectories with sufficiently low $\delta_{\boldsymbol{x}}(\Pi)$, then the library contains a policy that PL-CBF can certify as safe.

We define the *clearance* of a trajectory as its minimum safety margin:

Definition 5 (Trajectory Clearance)**.** *The clearance $\gamma(\sigma)$ of a trajectory σ over $[0, T]$ with respect to the constraint function h is*

$$\gamma(\sigma) \coloneqq \inf_{\tau \in [0,T]} h(\sigma(\tau)). \tag{16}$$

In particular, $\gamma(\varphi_{(\cdot)}^\pi(\boldsymbol{x})) = H_T^\pi(\boldsymbol{x})$ by (9)*.*

Theorem 1 (Completeness of a Finite-Horizon Safe Policy)**.** *Consider a system with a constraint function $h : \mathcal{X} \to \mathbb{R}$ that is L_h-Lipschitz. Let Π be a finite policy library with approximation precision $\delta_{\boldsymbol{x}}(\Pi)$ under the finite-horizon language metric $d_{\boldsymbol{x}}(\cdot, \cdot)$. Suppose there exists a safe policy $\pi^\star \in \Pi_{\text{all}}$ that renders the system strictly safe from $\boldsymbol{x}$ over $[0, T]$ with clearance $\gamma^\star = \gamma(\varphi_{(\cdot)}^{\pi^\star}(\boldsymbol{x})) > 0$. If*

$$\delta_{\boldsymbol{x}}(\Pi) < \gamma^\star / L_h, \tag{17}$$

then there exists $\pi_k \in \Pi$ such that $H_T^{\pi_k}(\boldsymbol{x}) > 0$.

Proof. By the definition of $\delta_{\boldsymbol{x}}(\Pi)$, there exists a policy $\pi_k \in \Pi$ such that

$$d_{\boldsymbol{x}}(\pi^\star, \pi_k) = \sup_{\tau \in [0,T]} \|\varphi_\tau^{\pi^\star}(\boldsymbol{x}) - \varphi_\tau^{\pi_k}(\boldsymbol{x})\|_2 \le \delta_{\boldsymbol{x}}(\Pi).$$

Hence, for all $\tau \in [0, T]$, $\|\varphi_\tau^{\pi^\star}(\boldsymbol{x}) - \varphi_\tau^{\pi_k}(\boldsymbol{x})\|_2 \le \delta_{\boldsymbol{x}}(\Pi)$. Since h is L_h-Lipschitz, for any $\tau \in [0, T]$,

$$\begin{aligned} &\left| h(\varphi_\tau^{\pi^\star}(\boldsymbol{x})) - h(\varphi_\tau^{\pi_k}(\boldsymbol{x})) \right| \\ \le\ & L_h \|\varphi_\tau^{\pi^\star}(\boldsymbol{x}) - \varphi_\tau^{\pi_k}(\boldsymbol{x})\|_2 \le L_h\, \delta_{\boldsymbol{x}}(\Pi). \end{aligned}$$

Rearranging gives, $\forall \tau \in [0, T]$,

$$h(\varphi_\tau^{\pi_k}(\boldsymbol{x})) \ge h(\varphi_\tau^{\pi^\star}(\boldsymbol{x})) - L_h \delta_{\boldsymbol{x}}(\Pi). \tag{18}$$

Taking the infimum over $\tau \in [0, T]$ in (18) yields

$$\begin{aligned} H_T^{\pi_k}(\boldsymbol{x}) &= \inf_{\tau \in [0,T]} h(\varphi_\tau^{\pi_k}(\boldsymbol{x})) \\ &\ge \inf_{\tau \in [0,T]} h(\varphi_\tau^{\pi^\star}(\boldsymbol{x})) - L_h \delta_{\boldsymbol{x}}(\Pi) = \gamma^\star - L_h \delta_{\boldsymbol{x}}(\Pi). \end{aligned}$$

If $\delta_{\boldsymbol{x}}(\Pi) < \gamma^\star / L_h$, then $\gamma^\star - L_h \delta_{\boldsymbol{x}}(\Pi) > 0$, and therefore $H_T^{\pi_k}(\boldsymbol{x}) > 0$. □

[3] A closely related finite-horizon construction for timed languages is referred to as a *timed language metric* in [20].

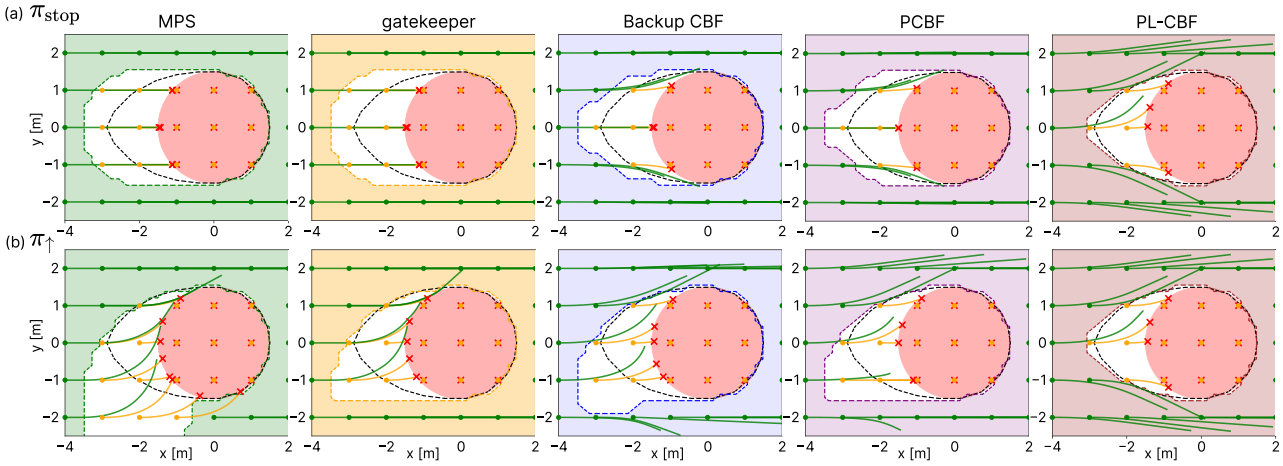


Fig. 1: Recovered safe sets for the double-integrator example on the slice $(v_x, v_y) = (2, 0)$. The black dashed line denotes the viability kernel computed via HJ reachability on the full 4-dim state space. For each 1 m $\times$ 1 m grid of initial positions, we simulate the closed-loop system under each safety filter: green trajectories are safe, while yellow trajectories correspond to unsafe or infeasible rollouts. (a) Top row: stopping fallback π_{stop}. (b) Bottom row: evasive fallback $\pi_{\uparrow}$. PL-CBF uses the same library in both rows, $\Pi = \{\pi_{\text{nom}}, \pi_{\text{stop}}, \pi_{\uparrow}, \pi_{\downarrow}\}$.

C. Relation to CBFs and Forward Invariance

Although PL-CBF uses a finite-horizon rollout value, the discussion in this subsection is purely conceptual and considers the idealized infinite-horizon limit.

For a fixed policy π, define the infinite-horizon policy value function as $H^{\pi}_{\infty}(\boldsymbol{x}) := \inf_{t\geq 0} h(\varphi^{\pi}_t(\boldsymbol{x}))$. By definition, $H^{\pi}_{\infty}(\boldsymbol{x}) \leq h(\boldsymbol{x})$ for all $\boldsymbol{x}$. Moreover, H^{π}_{∞} is non-decreasing along the closed-loop trajectory induced by π, satisfying the monotonicity condition:

$$\nabla H^{\pi}_{\infty}(\boldsymbol{x})^{\top} \left(f(\boldsymbol{x}) + g(\boldsymbol{x})\pi(\boldsymbol{x})\right) \geq 0. \tag{19}$$

These properties are equivalent to H^{π}_{∞} satisfying a Hamilton-Jacobi variational inequality in the viscosity sense [9]:

$$\begin{aligned} \min \Big\{ & h(\boldsymbol{x}) - H^{\pi}_{\infty}(\boldsymbol{x}), \\ & \nabla H^{\pi}_{\infty}(\boldsymbol{x})^{\top} \left(f(\boldsymbol{x}) + g(\boldsymbol{x})\pi(\boldsymbol{x})\right) \Big\} = 0. \end{aligned} \tag{20}$$

Consequently, H^{π}_{∞} is a CBF on the set $\mathcal{S}^{\pi}_{\infty}$, where $\mathcal{S}^{\pi}_{\infty} := \{\boldsymbol{x} \in \mathcal{X} \mid H^{\pi}_{\infty}(\boldsymbol{x}) \geq 0\}$ [13, Thm 1].

For a policy library Π, define $H^{\Pi}_{\infty}(\boldsymbol{x}) := \max_{\pi_i \in \Pi} H^{\pi_i}_{\infty}(\boldsymbol{x})$. Since the pointwise maximum of multiple CBFs is a CBF by utilizing nonsmooth analysis and generalized gradients in [22], H^{Π}_{∞} is also a CBF.

Finally, if the library contains a backup policy π_{b} (Def. 2), i.e., $\pi_{\text{b}} \in \Pi$, then the PL-CBF safety filter recovers the infinite-time safety (forward invariance) and recursive feasibility properties associated with backup-based safety filters.

D. Inclusion of Nominal Policy

Many backup-based safety filters often exhibit unnecessary conservatism by modifying the nominal control even when the system is operating safely [12]–[14], [21]. This phenomenon, which we refer to as "safety evaluation on backup" [23], occurs because safety is typically evaluated based on the feasibility of a specific backup maneuver rather than the nominal plan itself. If the backup policy is inherently conservative or currently ill-suited to the environment, the filter may restrict the nominal controller unnecessarily to ensure the backup remains feasible.

The PL-CBF addresses this by explicitly including the nominal policy π_{nom} in the library Π. Consider a scenario where pre-defined fallback policies are only marginally feasible or unsafe, yet the nominal plan remains safe, and $\pi_{\text{nom}} \in \Pi$. In this case, the PL-CBF selects π_{nom} as the least invasive policy. The resulting safety constraint $K_{\text{cbf}}(\boldsymbol{x}; \pi_{\text{nom}})$ naturally contains the nominal control input $\boldsymbol{u}_{\text{nom}} = \pi_{\text{nom}}(\boldsymbol{x})$. Consequently, the solution to the QP in (13) yields $\boldsymbol{u}^{\star} = \boldsymbol{u}_{\text{nom}}$, resulting in zero intervention.

VI. RESULTS

We evaluate PL-CBF as a runtime safety filter in three simulation case studies of increasing difficulty. We compare against four representative safety filters using a single fallback (or backup) policy: Model Predictive Shielding (MPS) [21], `gatekeeper` [2], Backup CBF [12], and the Policy CBF (PCBF; Sec. II-B) [14]. Across all experiments, methods share the same nominal controller and differ only in (i) the safety-filter's algorithm and (ii) the fallback policy. All experiments were conducted on a MacBook Air with the Apple M4 CPU.

A. Safe Set Under Multiple Fallback Policies

We first illustrate set-wise behavior in a planar double-integrator example where a reference viability kernel can be computed. Let $\boldsymbol{x} = [\boldsymbol{p}^{\top}\ \boldsymbol{v}^{\top}]^{\top} \in \mathbb{R}^4$ with $\boldsymbol{p} = [x\ y]^{\top}$ and $\boldsymbol{v} = [v_x\ v_y]^{\top}$, and dynamics

$$\dot{\boldsymbol{p}} = \boldsymbol{v}, \quad \dot{\boldsymbol{v}} = \boldsymbol{u}, \quad \boldsymbol{u} \in [-a_{\max}, a_{\max}]^2, \tag{21}$$

with $a_{\max} = 0.5$ m/s^2. The unsafe set is a disk $\mathcal{O} = \{\boldsymbol{x} \mid \|\boldsymbol{p}\|_2 \leq r_{\text{obs}}\}$, and we use $h(\boldsymbol{x}) = \|\boldsymbol{p}\|_2 - r_{\text{obs}}$, yielding $\mathcal{C} = \{\boldsymbol{x} \mid h(\boldsymbol{x}) \geq 0\}$. Since the state dimension is 4, we can compute the viability kernel for (21) using HJ reachability [24] and visualize it on the velocity slice $(v_x, v_y) = (2, 0)$ in m/s.

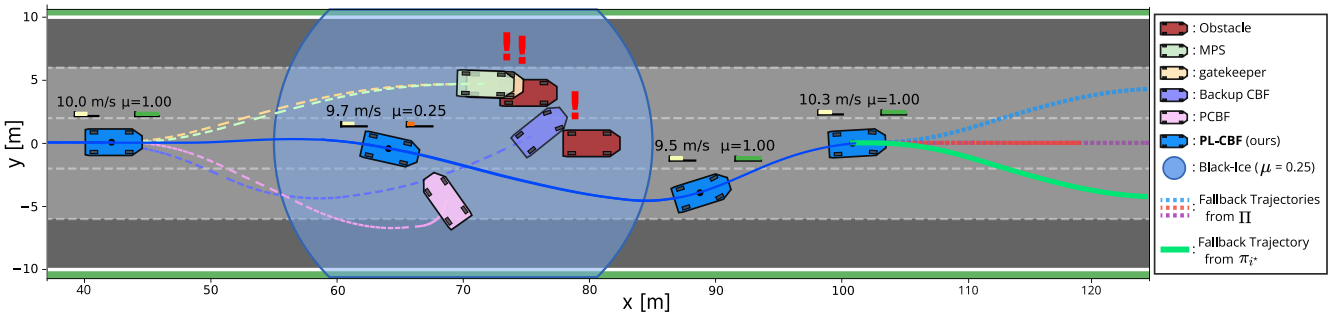


Fig. 2: Representative trajectory from the highway driving experiment in Table I with MPCC reference speed $v_{\text{ref}} = 10$ m/s. The four single-fallback baselines are shown with the left lane-change controller π_{left}: MPS, `gatekeeper`, and Backup CBF collide with the obstacle, while PCBF undergoes an oversteer-induced spin after entering the black-ice patch ($\mu = 0.25$). PL-CBF safely traverses the low-friction segment by selecting from $\Pi = \{\pi_{\text{stop}}, \pi_{\text{left}}, \pi_{\text{right}}, \pi_{\text{nom}}\}$. Near $x \approx 100$ m, dashed lines show candidate rollouts from Π; the purple dashed curve is the MPCC nominal rollout, and the green curve is the selected rollout from $\pi_{i^\star}$.

To highlight the dependence of single-policy certificates on the chosen maneuver, we consider two simple fallbacks: a stopping policy π_{stop} implemented as component-wise saturated full deceleration until rest, i.e., $-a_{\max}\,\text{sign}(\boldsymbol{v})$, and a "turn-up" evasive policy $\pi_{\uparrow}$ that applies saturated acceleration in $+y$. Fig. 1 compares the certified safe sets obtained under these two fallback choices. PL-CBF uses the same library in both rows, $\Pi = \{\pi_{\text{nom}}, \pi_{\text{stop}}, \pi_{\uparrow}, \pi_{\downarrow}\}$, where $\pi_{\downarrow}$ applies saturated acceleration in $-y$.

Importantly, a single-policy method may recover a different safe set if a different fallback is chosen, but each fixed fallback certifies only the states compatible with that particular maneuver. As a result, replacing the stopping fallback with a different maneuver may recover states that require that specific behavior while losing those that are safe only under stopping. PL-CBF avoids this tradeoff by evaluating a finite library online and certifying safety whenever at least one library policy remains feasible over the horizon, yielding the largest safe set.

B. Highway Driving with Sudden Friction Change

We next consider a highway driving scenario where the road friction μ is revealed online. We adopt a realistic vehicle model with a Fiala brush tire model [25], which has been experimentally demonstrated to accurately model vehicle behavior even in extreme situations such as racing and drifting [26]. Including the global coordinates, the model has an 8-dim state and a 2-dim input:

$$\boldsymbol{x} = [\ p_x\ p_y\ \psi\ r\ \beta\ V\ \delta\ \tau\ ]^\top, \qquad \boldsymbol{u} = [\ \dot{\delta}\ \dot{\tau}\ ]^\top, \tag{22}$$

where (p_x, p_y) is the planar position of the center of mass (CoM), ψ is yaw, r is yaw rate, β is sideslip, V is speed, δ is steering angle, and τ is rear-wheel torque. The kinematics and dynamics are

$$\begin{aligned}
&\dot{p}_x = V\cos(\psi+\beta), \quad \dot{p}_y = V\sin(\psi+\beta), \quad \dot{\psi} = r, \\
&\dot{r} = \frac{a(F_{x,\text{f}}\sin\delta + F_{y,\text{f}}\cos\delta) - bF_{y,\text{r}}}{I_z}, \\
&\dot{\beta} = \frac{F_{x,\text{f}}\sin(\delta-\beta) + F_{y,\text{f}}\cos(\delta-\beta) - F_{x,\text{r}}\sin\beta + F_{y,\text{r}}\cos\beta}{mV} - r, \\
&\dot{V} = \frac{F_{x,\text{f}}\cos(\delta-\beta) - F_{y,\text{f}}\sin(\delta-\beta) + F_{x,\text{r}}\cos\beta + F_{y,\text{r}}\sin\beta}{m},
\end{aligned} \tag{23}$$

TABLE I: Highway driving with sudden friction change. **Fail@10 m/s**: collision/infeasibility over 50 trials at $v_{\text{ref}} = 10$ m/s. $v_{\text{ref}}^{\max}$: largest target speed that passes all benchmark configurations in an offline worst-case-clearance analysis at the friction transition. For MPS and `gatekeeper`, lane-change policies are not applicable backups due to their fundamental mechanisms of binary nominal/backup switching. **Time** is reported as the average per-step safety-filter solve time (averaged over fallback choices for single-fallback baselines).

Algorithm	Fallback	Fail@10 m/s	$v_{\text{ref}}^{\max}$ [m/s]	Time [ms]
MPS [21]	π_{stop}	40/50 (80.0%)	2.50	2.612
	π_{left}	36/50 (72.0%)	N/A	
	π_{right}	43/50 (86.0%)	N/A	
`gatekeeper` [2]	π_{stop}	40/50 (80.0%)	2.75	4.143
	π_{left}	36/50 (72.0%)	N/A	
	π_{right}	46/50 (92.0%)	N/A	
Backup CBF [12]	π_{stop}	30/50 (60.0%)	2.75	24.789
	π_{left}	14/50 (28.0%)	5.00	
	π_{right}	16/50 (32.0%)	5.00	
PCBF [14]	π_{stop}	40/50 (80.0%)	2.75	5.286
	π_{left}	18/50 (36.0%)	4.50	
	π_{right}	11/50 (22.0%)	5.00	
PL-CBF	Π	**0/50 (0.0%)**	**10.00**	7.522

Slip angles are $\alpha_{\text{f}} = \arctan\left(\frac{V\sin\beta + ar}{V\cos\beta}\right) - \delta$, $\alpha_{\text{r}} = \arctan\left(\frac{V\sin\beta - br}{V\cos\beta}\right)$, where a and b are the distances between the CoM and front and rear axles. m is the mass, and I_z is the moment of inertia about CoM. Lateral forces are given by the Fiala brush model, where C_{c} denotes the cornering stiffness:

$$F_y = \begin{cases} -C_{\text{c}}\tan\alpha + \frac{C_{\text{c}}^2}{3F_{y,\max}}|\tan\alpha|\tan\alpha - \frac{C_{\text{c}}^3}{27F_{y,\max}^2}\tan^3\alpha & \\ \qquad\qquad\qquad\qquad\qquad\qquad \text{if} \quad |\alpha| < \alpha_{\text{sl}}, & \\ -F_{y,\max}\,\text{sgn}(\alpha) \qquad\qquad \text{if} \quad |\alpha| > \alpha_{\text{sl}}, & \end{cases} \tag{24}$$

with $\alpha_{\text{sl}} = \text{atan}\left(\frac{3F_{y,\max}}{C_{\text{c}}}\right)$ and $F_{y,\max} = \sqrt{(\mu F_z)^2 - \gamma F_x^2}$, where F_z and F_x are normal and longitudinal force. μ is the road friction. Normal loads follow the static distribution $F_{z,\text{f}} = \frac{mgb}{a+b}$ and $F_{z,\text{r}} = \frac{mga}{a+b}$. To model longitudinal slip and friction-limited braking, we saturate the rear longitudinal force using a smooth friction-limited model:

$$F_{x,\text{f}} = 0, \qquad F_{x,\text{r}} = (\mu F_{z,\text{r}})\tanh\left(\frac{\tau}{r_{\text{w}}(\mu F_{z,\text{r}})}\right), \tag{25}$$

where r_{w} is the wheel radius. Therefore, both longitudinal and lateral forces depend nonlinearly on the road friction

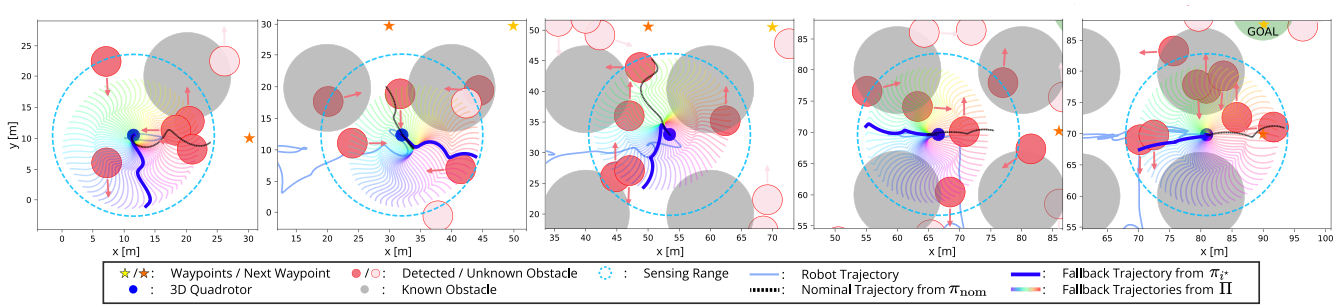


Fig. 3: Warehouse navigation using PL-CBF with $P = 64$. The robot tracks sequential waypoints (stars) while reacting to newly sensed dynamic obstacles. At each step, PL-CBF selects the least-restrictive fallback from its library (thick blue line) and applies the QP-based safety filter online (Alg. 1).

μ. In this experiment, the friction coefficient changes across road segments and both the controller and the safety filters use the currently available friction estimate online.

Scenario. The nominal controller is a Model Predictive Contouring Controller (MPCC) [27] that tracks the centerline of the current lane at target speed v_{ref}. Obstacles are detected only within a limited sensing range, and the friction estimate μ is updated only after the vehicle enters the corresponding road segment. We place a black-ice patch with $\mu = 0.25$ near the obstacles, while the remainder of the road has $\mu = 1.0$. We randomly sample one or two obstacles within the low-friction patch, constrained to occupy distinct lanes of the three-lane roadway. The policy library is $\Pi = \{\pi_{\text{stop}}, \pi_{\text{left}}, \pi_{\text{right}}, \pi_{\text{nom}}\}$, where π_{stop} applies maximum deceleration, π_{left} and π_{right} are tuned feedback lane-change controllers to the target lane, and π_{nom} is the MPCC. Under nominal friction and sufficient sensing, π_{stop} is a valid backup; however, when an obstacle lies on or beyond the low-friction segment, braking alone may no longer be valid under the input constraints and can lead to collision because the reduced friction envelope in (24)–(25) limits the available tire forces.

Results. With $v_{\text{ref}} = 10$ m/s, PL-CBF achieves $0/50$ failures, whereas all single-fallback baselines incur collision or infeasibility (Table I). Although estimating changing dynamics quantities online is outside the scope of this paper and is left to future work, this experiment shows that PL-CBF has clear merit when the operating condition changes and a single backup maneuver can lose validity. To quantify how conservative a single fallback must be to remain uniformly safe, we report $v_{\text{ref}}^{\max}$ from an offline worst-case-clearance analysis at the friction transition: PL-CBF remains safe up to the maximum tested 10 m/s, while single-fallback baselines require substantially smaller reference speeds. A representative trial is shown in Fig. 2. Despite evaluating multiple policies, PL-CBF maintains low runtime latency of **7.5 ms per step**.

C. 3D Quadrotor Navigation in Crowded Environments

Finally, we evaluate a crowded warehouse navigation task with limited sensing and dynamic obstacles. The robot is tasked with visiting waypoints sequentially while avoiding both known static obstacles (gray) and newly detected dynamic obstacles (red). We use a linearized 3D quadrotor model with 12 states and 4 inputs [28]:

$$\boldsymbol{x} = [\, x \;\; y \;\; z \;\; \dot{x} \;\; \dot{y} \;\; \dot{z} \;\; \phi \;\; \theta \;\; \psi \;\; \dot{\phi} \;\; \dot{\theta} \;\; \dot{\psi} \,]^\top, \;\; \boldsymbol{u} = [\, \tau_\phi \;\; \tau_\theta \;\; \tau_\psi \;\; f_z \,]^\top,$$

with equations of motion

$$\ddot{x} = g\theta, \;\; \ddot{\theta} = \frac{L}{I_y}\tau_\theta, \;\; \ddot{y} = -g\phi, \ddot{\phi} = \frac{L}{I_x}\tau_\phi, \;\; \ddot{z} = \frac{f_z - mg}{m}, \;\; \ddot{\psi} = \frac{1}{I_z}\tau_\psi.$$

The moments of inertia I_x, I_y, and I_z are defined about the body-fixed axes. We impose realistic physical limits, including a thrust-to-weight ratio of 2.36. This model is particularly challenging for classical CBF synthesis because (a) the state dimension is high ($\mathbb{R}^{12}$), and (b) for geometric constraints defined on position, the relative degree with respect to $\boldsymbol{u}$ is four.

TABLE II: 3D quadrotor navigation (100 trials). Single-policy baselines use the same retrace fallback. PL-CBF ablation varies the number of evasive maneuvers $P \in \{4, 8, 16, 32, 64\}$ with headings uniformly spaced on $[0, 2\pi)$, plus π_{nom} (thus $|\Pi| = P + 1$). Compute time is reported as the average per-step safety-filter solve time.

Algorithm	P	Collision/Infeasible	Compute Time [ms]
MPS	–	77/100 (77.0%)	5.605
gatekeeper	–	68/100 (68.0%)	32.257
Backup CBF	–	74/100 (74.0%)	96.621
PCBF	–	79/100 (79.0%)	15.474
MI-MPC	64	73/100 (73.0%)	688.230
PL-CBF	4	8/100 (8.0%)	15.043
PL-CBF	8	6/100 (6.0%)	15.503
PL-CBF	16	5/100 (5.0%)	16.989
PL-CBF	32	3/100 (3.0%)	19.886
PL-CBF	64	**0/100 (0.0%)**	27.986

Scenario. The environment contains 16 known static obstacles and 45 dynamic obstacles that are initially unobserved and become available only within the sensing range. We evaluate 100 randomized trials by sampling the dynamic obstacles' initial positions and velocities. All single-policy baselines use the same retrace fallback that commands the robot toward the previous waypoint, a common heuristic in receding-horizon navigation. We ablate PL-CBF by varying the number of evasive maneuvers $P \in \{4, 8, 16, 32, 64\}$, implemented as PD controllers with heading directions uniformly spaced on $[0, 2\pi)$, together with the nominal waypoint-tracking policy, yielding $|\Pi| = P + 1$.

MI-MPC baseline. As an optimization-based baseline that enforces a "one-of-many" maneuver selection within planning, we implement a mixed-integer MPC (MI-MPC)

that selects exactly one candidate maneuver rollout using binary variables and mixed-integer linear constraints. We use the same maneuver set as PL-CBF with $P = 64$ and follow a standard MI-MPC formulation as described in [29].

Results. Table II reports failures and computation time. Single-policy baselines fail frequently, indicating that reliance on a single retrace maneuver can be overly restrictive in dense dynamic environments. Increasing the PL-CBF library size improves safety monotonically, consistent with richer maneuver coverage. Fig. 3 shows a representative behavior where PL-CBF selects a least-restrictive fallback online and solves the QP at each step. MI-MPC is substantially slower (688.2 ms), reflecting the cost of enforcing disjunctive constraints online.

VII. CONCLUSION

We presented PL-CBF, a runtime safety filter for systems with evolving perceived safety constraints. PL-CBF retains multimodal safe maneuvers through a library of fallback policies, and selects the least invasive safe mode at each time step by evaluating the library online via parallelizable finite-horizon rollouts. The resulting structure makes PL-CBF practical as a deployable high-rate safety layer in unstructured environments where reliance on a single fixed fallback maneuver is brittle. Future work will focus on systematic policy library construction using learning-based approaches, handling of model mismatch and perception uncertainty, and adaptive mechanisms that refine the library online without sacrificing real-time performance.

REFERENCES

[1] B. Zhou, J. Pan, F. Gao, and S. Shen, "RAPTOR: Robust and Perception-Aware Trajectory Replanning for Quadrotor Fast Flight," *IEEE Transactions on Robotics*, vol. 37, no. 6, pp. 1992–2009, 2021.

[2] D. R. Agrawal, R. Chen, and D. Panagou, "gatekeeper: Online Safety Verification and Control for Nonlinear Systems in Dynamic Environments," *IEEE Transactions on Robotics*, vol. 40, pp. 4358–4375, 2024.

[3] T. Kim and D. Panagou, "Visibility-Aware RRT* for Safety-Critical Navigation of Perception-Limited Robots in Unknown Environments," *IEEE Robotics and Automation Letters*, vol. 10, no. 5, pp. 4508–4515, 2025.

[4] R. Firoozi, A. Mir, G. S. Camps, and M. Schwager, "OA-MPC: Occlusion-Aware MPC for Guaranteed Safe Robot Navigation With Unseen Dynamic Obstacles," *IEEE Transactions on Control Systems Technology*, vol. 33, no. 3, pp. 940–951, 2025.

[5] H. K. Park, T. Kim, and D. Panagou, "Beyond Collision Cones: Dynamic Obstacle Avoidance for Nonholonomic Robots via Dynamic Parabolic Control Barrier Functions," in *International Conference on Robotics and Automation (ICRA)*, 2026.

[6] N. A. Spielberg, M. Brown, and J. C. Gerdes, "Neural Network Model Predictive Motion Control Applied to Automated Driving With Unknown Friction," *IEEE Transactions on Control Systems Technology*, vol. 30, no. 5, pp. 1934–1945, 2022.

[7] T. Kim, H. Lee, and W. Lee, "Physics Embedded Neural Network Vehicle Model and Applications in Risk-Aware Autonomous Driving Using Latent Features," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2022, pp. 4182–4189.

[8] S. Yang, M. Black, G. Fainekos, B. Hoxha, H. Okamoto, and R. Mangharam, "Safe Control Synthesis for Hybrid Systems through Local Control Barrier Functions," in *American Control Conference (ACC)*, 2024, pp. 344–351.

[9] A. Altarovici, O. Bokanowski, and H. Zidani, "A general Hamilton-Jacobi framework for non-linear state-constrained control problems," *ESAIM: Control, Optimisation and Calculus of Variations*, vol. 19, no. 2, pp. 337–357, 2013.

[10] A. D. Ames, S. Coogan, M. Egerstedt, G. Notomista, K. Sreenath, and P. Tabuada, "Control Barrier Functions: Theory and Applications," in *European Control Conference (ECC)*, 2019, pp. 3420–3431.

[11] K. Garg, J. Usevitch, J. Breeden, M. Black, D. Agrawal, H. Parwana, and D. Panagou, "Advances in the Theory of Control Barrier Functions: Addressing practical challenges in safe control synthesis for autonomous and robotic systems," *Annual Reviews in Control*, vol. 57, p. 100945, 2024.

[12] Y. Chen, M. Jankovic, M. Santillo, and A. D. Ames, "Backup Control Barrier Functions: Formulation and Comparative Study," in *IEEE Conference on Decision and Control (CDC)*, 2021, pp. 6835–6841.

[13] O. So, Z. Serlin, M. Mann, J. Gonzales, K. Rutledge, N. Roy, and C. Fan, "How to Train Your Neural Control Barrier Function: Learning Safety Filters for Complex Input-Constrained Systems," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2024, pp. 11 532–11 539.

[14] L. Knoedler, O. So, J. Yin, M. Black, Z. Serlin, P. Tsiotras, J. Alonso-Mora, and C. Fan, "Safety on the Fly: Constructing Robust Safety Filters via Policy Control Barrier Functions at Runtime," *IEEE Robotics and Automation Letters*, vol. 10, no. 10, pp. 10 058–10 065, 2025.

[15] E. F. Camacho and C. Bordons, *Model Predictive Control*. Springer London, 2004.

[16] L. D. Re, F. Allgöwer, L. Glielmo, C. Guardiola, and I. Kolmanovsky, *Automotive Model Predictive Control: Models, Methods and Applications*. Springer, 2010.

[17] J. Bradbury, R. Frostig, P. Hawkins, M. J. Johnson, C. Leary, D. Maclaurin, G. Necula, A. Paszke, J. VanderPlas, S. Wanderman-Milne, and Q. Zhang, "JAX: composable transformations of Python+NumPy programs," 2018.

[18] A. Girard and G. J. Pappas, "Approximation Metrics for Discrete and Continuous Systems," *IEEE Transactions on Automatic Control*, vol. 52, no. 5, pp. 782–798, 2007.

[19] ——, "Approximate Bisimulation: A Bridge Between Computer Science and Control Theory," *European Journal of Control*, vol. 17, no. 5, pp. 568–578, 2011.

[20] Y. Deng, A. D'Innocenzo, M. D. Di Benedetto, S. Di Gennaro, and A. A. Julius, "Verification of Hybrid Automata Diagnosability With Measurement Uncertainty," *IEEE Transactions on Automatic Control*, vol. 61, no. 4, pp. 982–993, 2016.

[21] O. Bastani, "Safe Reinforcement Learning with Nonlinear Dynamics via Model Predictive Shielding," in *American Control Conference (ACC)*, 2021, pp. 3488–3494.

[22] P. Glotfelter, J. Cortés, and M. Egerstedt, "Boolean Composability of Constraints and Control Synthesis for Multi-Robot Systems via Non-smooth Control Barrier Functions," in *IEEE Conference on Control Technology and Applications (CCTA)*, 2018, pp. 897–902.

[23] T. Kim, A. D. Menon, A. Trivedi, and D. Panagou, "Backup-Based Safety Filters: A Comparative Review of Backup CBF, Model Predictive Shielding, and gatekeeper," in *arXiv preprint arXiv:2604.02401*, 2026.

[24] S. Bansal, M. Chen, S. Herbert, and C. J. Tomlin, "Hamilton-Jacobi reachability: A brief overview and recent advances," in *IEEE Conference on Decision and Control (CDC)*, 2017, pp. 2242–2253.

[25] J. Y. M. Goh, M. Thompson, J. Dallas, and A. Balachandran, "Beyond the stable handling limits: nonlinear model predictive control for highly transient autonomous drifting," *Vehicle System Dynamics*, vol. 62, no. 10, pp. 2590–2613, 2024.

[26] J. Dallas, J. Talbot, M. Suminaka, M. Thompson, T. Lew, G. Orosz, and J. Subosits, "Control Barrier Functions for Shared Control and Vehicle Safety," in *American Control Conference (ACC)*, 2025, pp. 4203–4210.

[27] D. Lam, C. Manzie, and M. Good, "Model predictive contouring control," in *IEEE Conference on Decision and Control (CDC)*, 2010, pp. 6137–6142.

[28] A. M. Annaswamy, A. Guha, Y. Cui, S. Tang, P. A. Fisher, and J. E. Gaudio, "Integration of Adaptive Control and Reinforcement Learning for Real-Time Control and Learning," *IEEE Transactions on Automatic Control*, vol. 68, no. 12, pp. 7740–7755, 2023.

[29] F. Borrelli, A. Bemporad, and M. Morari, *Predictive Control for Linear and Hybrid Systems*. Cambridge University Press, 2017.